

AI 使用说明

1. 使用的 AI 模型

本次挑战主要使用 ChatGPT/Codex 的 GPT 系列模型辅助学习、排查环境问题和整理材料。现有记录没有保留具体模型版本，因此不填写无法核实的型号。AI 作为检索、解释和代码协作工具使用，最终的运行结果、论文内容和提交材料由我本人检查确认。

2. 使用过的提示词类型

使用过的提问方向如下。以下是按实际任务整理的提示词示例，并非逐字保存的聊天记录：

- “请解释 SGLang 的 RadixAttention 和 RadixCache 分别解决什么问题，并说明它们在一次请求流程中的位置。”
- “请对比 vLLM PagedAttention、SGLang RadixAttention 和 paged KV cache 的联系与区别。”
- “请根据 SGLang/RadixAttention 论文和 PagedAttention 论文，整理与 KV Cache、Prefix Reuse 相关的概念。”
- “请检查这个环境配置/报错，按依赖、服务启动、HTTP 接口三个层次给出排查步骤。”
- “请根据 workload 的 input length 和 output length 设计 synthetic prompt，并记录每条请求的状态和延迟。”
- “请把实验过程整理成适合作业提交的复盘材料，区分事实、解释和个人认识。”

这些提示词通常会补充具体版本、命令输出、论文段落或实验结果，使回答能够针对当前问题，而不是只得到泛化介绍。

3. AI 如何帮助学习，以及在哪些环节使用

AI 主要帮助了四个环节。第一，在环境配置阶段，AI 将 SGLang、Ray、模型和接口要求拆成检查清单，帮助定位启动失败或依赖不匹配。第二，在论文阅读阶段，AI 用较直观的语言解释 Prefill、Decode、KV Cache、Radix Tree、PagedAttention 等术语，并协助建立概念之间的对照关系。第三，在实验阶段，AI 帮助设计 workload 生成和结果记录逻辑，检查 JSONL 字段和延迟统计是否合理。第四，在整理阶段，AI 协助把实验步骤、理论回答和个人反思组织成结构清晰的文档。

AI 对学习的帮助不只是直接给答案，更重要的是提供了问题拆解方式。例如把“理解 RadixAttention”拆成“缓存什么、如何匹配、何时复用、如何淘汰、对 Prefill 有什么影响”几个问题后，我能更有针对性地回到论文和代码中寻找证据。

4. 是否出现过 AI 误导，以及如何验证

出现过需要警惕的情况。AI 有时会把不同框架的术语用相近说法概括，或者把论文中的实现细节说成所有版本都完全如此；在环境问题上，也可能给出适用于其他 CUDA、Python 或 SGLang 版本的命令。因此，我没有把 AI 的回答直接当作实验结论。

我的验证方法是：查 SGLang 和 vLLM 官方文档及原始论文，并用 /v1/models、实际推理请求、workload JSONL 和截图核对。回查历史代码后发现，原文“保持长度分布”和“按 Poisson 到达过程安排请求”与实际执行不符：旧输入固定为 3000 个客户端 token，旧请求为串行处理后指数等待。因此已修正文档，保留历史原始数据和失败记录，不将新脚本当作已完成实验。新一轮次已保存实际环境版本输出：SGLang 0.5.14、Ray 2.56.0。论文中的缓存优化机制用于解释服务流程，现有记录不能验证缓存收益。这说明 AI 的解释和整理必须接受代码与数据核验。

修正实验与核验

修正实验已实际完成：从输入 1-3000 token、输出 1-512 token 的 2708 条候选 trace 中，用随机种子 20261008 采样 10 条，保留各条原始输入长度与输出上限。独立累计指数间隔生成 Poisson 到达时间， $\lambda=0.5$ 请求/秒；客户端按计划发送，不等待上一请求完成。10/10 请求成功，平均 latency 为 1.3982 秒，最大调度偏差为 4.38 毫秒。实际输出允许因 EOS 提前结束，已记录 finish_reason。服务输入计数与本地 chat template 一致，均比合成正文多 8 token；此核验仅适用于新一轮次。没有测量 TTFT 或缓存性能，不能据此宣称论文优化收益。

首次重跑因默认 FlashInfer 采样后端缺少 nvcc 而失败；改用 PyTorch 采样后端后重试成功。失败轮独立保留，不混入成功统计。单次接口推理返回 HTTP 200，只证明调用链路可用，不证明回答内容正确。